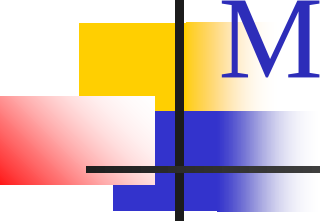


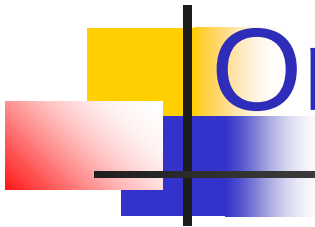
SOFTMAX REGRESSION

Dr. Nguyen Thi Kim Ngan



Multi-class classification

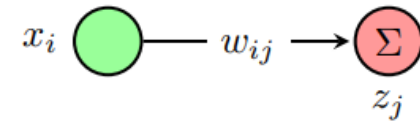
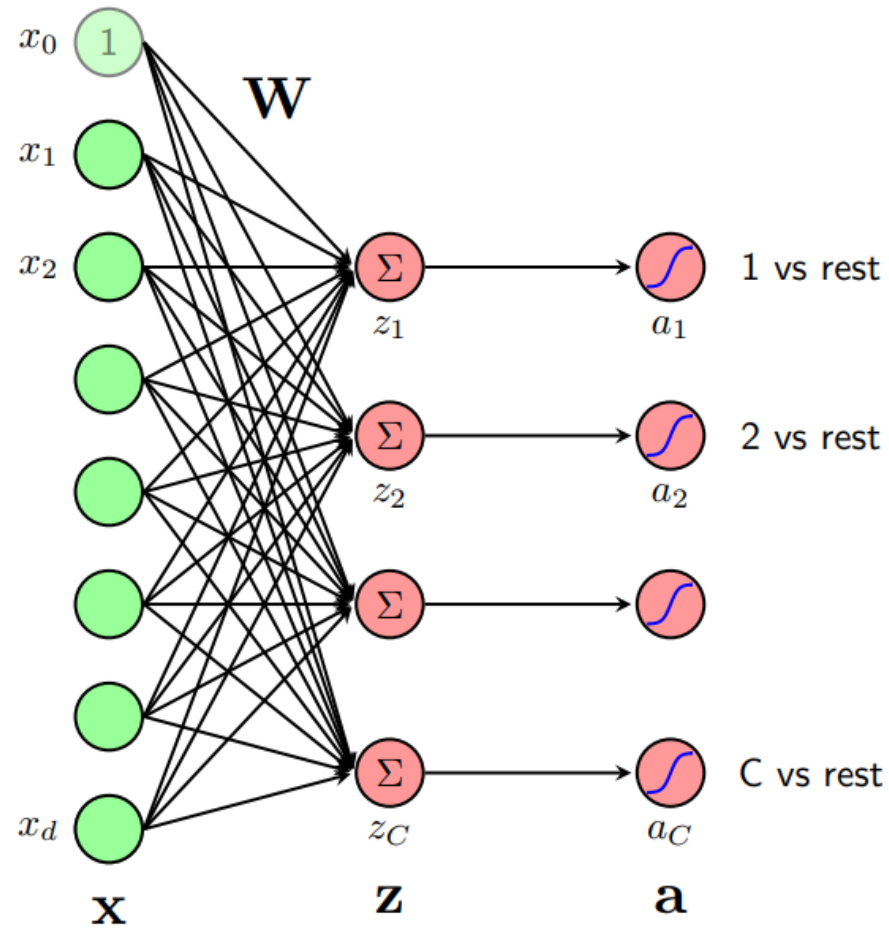
- **Input:** A labeled dataset $(X_{\text{train}}, y_{\text{train}})$, where the label of sample x_i is y_i , and y_i takes integer values in the range $[1, C]$
- **Output:** A classification model



One-vs-Rest

- Construct C classification models
- The i -th classification model ($i = 1, \dots, C$) considers a binary classification problem: class labeled i versus all other classes. That is the label set y_{train} consists of two labels: label i and label not i

Logistic Regression and One-vs-Rest



w_{0j} : biases, don't forget!

d : data dimension

C : number of classes

$\mathbf{x} \in \mathbb{R}^{d+1}$

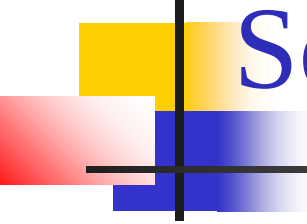
$\mathbf{W} \in \mathbb{R}^{(d+1) \times C}$

$z_i = \mathbf{w}_i^T \mathbf{x}$

$\mathbf{z} = \mathbf{W}^T \mathbf{x} \in \mathbb{R}^C$

$a_i = \text{sigmoid}(z_i) \in \mathbb{R}$

$0 < a_i < 1$



Softmax function

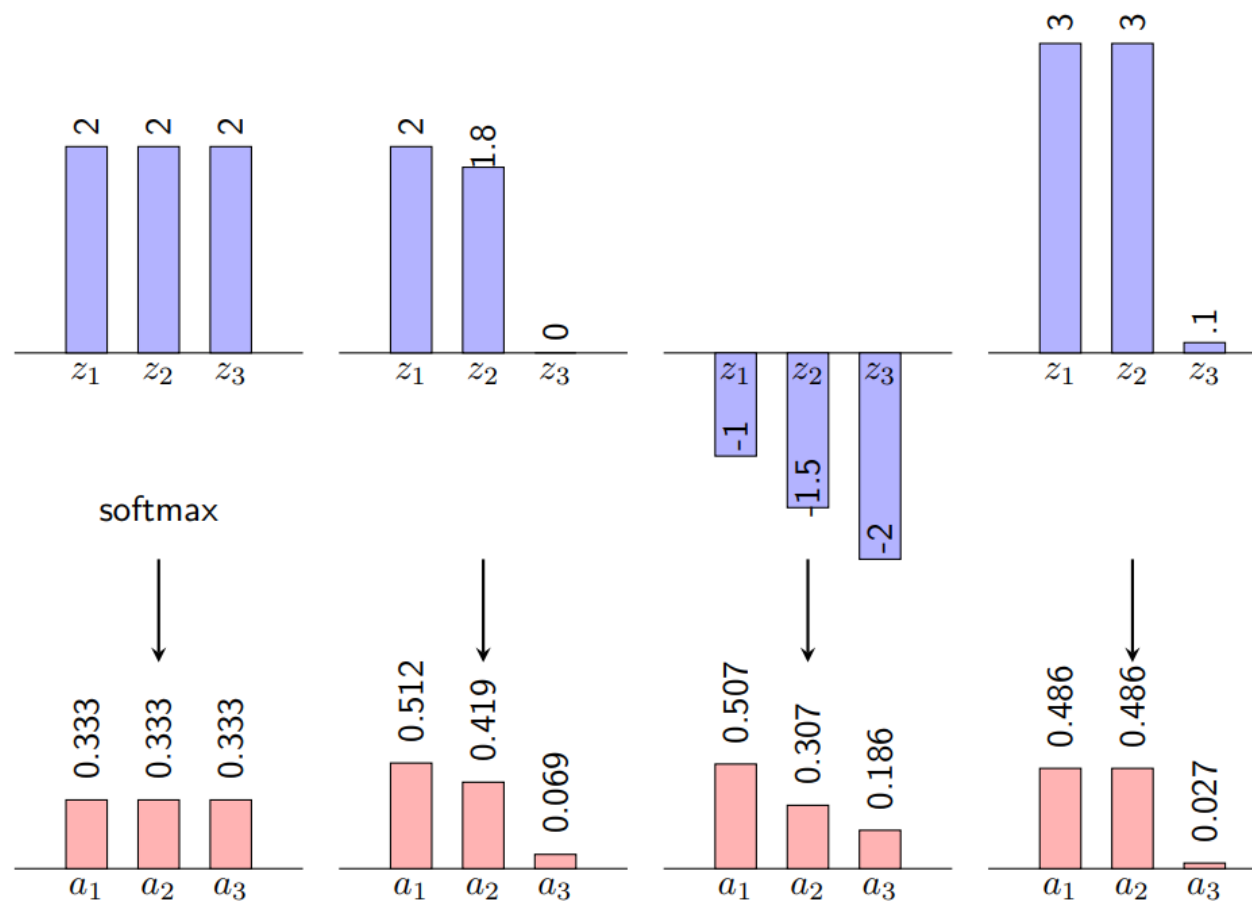
We need to find a differentiable, positive, and monotonically increasing function such that the predicted probabilities sum to 1

$$z_i = \mathbf{w}_i^T \mathbf{x}$$

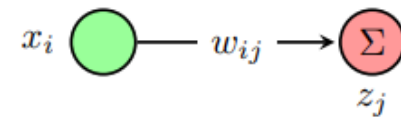
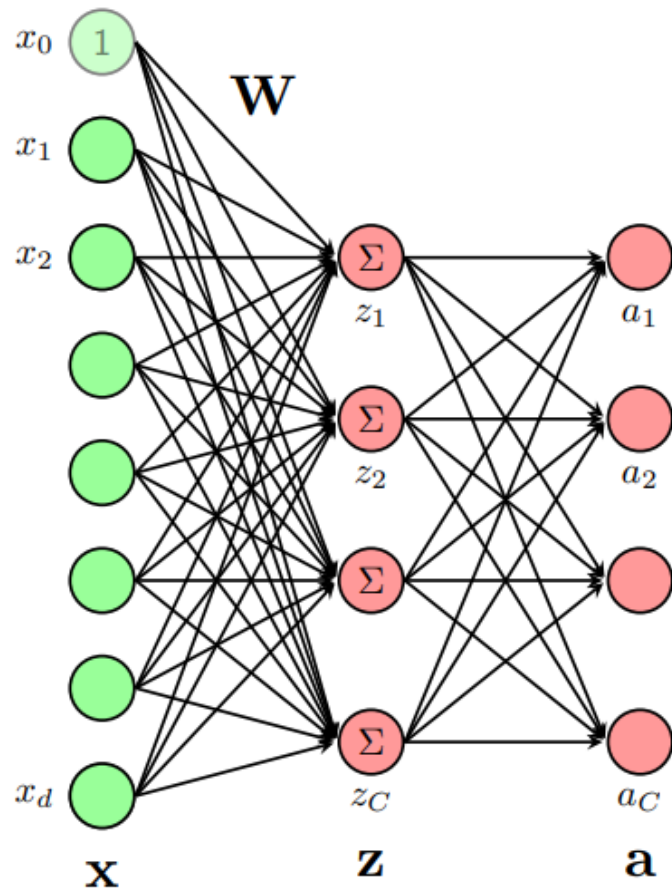
$$a_i = \frac{\exp(z_i)}{\sum_{j=1}^C \exp(z_j)}, \quad \forall i = 1, 2, \dots, C$$

$$p(y_k = i | \mathbf{x}_k; \mathbf{W}) = a_i$$

Examples of the input and output of Softmax



Softmax Regression Architecture



w_{0j} : biases, don't forget!

d : data dimension

C : number of classes

$\mathbf{x} \in \mathbb{R}^{d+1}$

$\mathbf{W} \in \mathbb{R}^{(d+1) \times C}$

$z_i = \mathbf{w}_i^T \mathbf{x}$

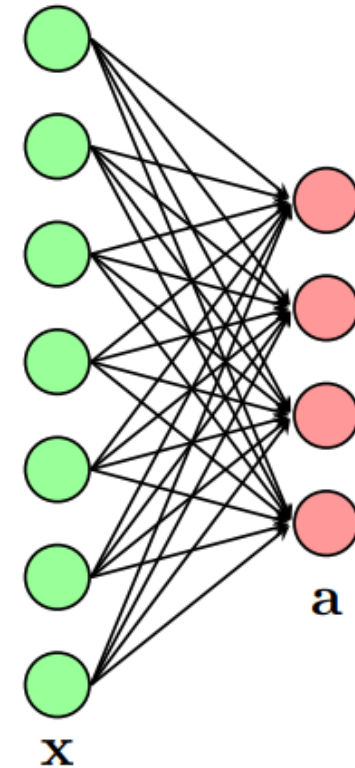
$\mathbf{z} = \mathbf{W}^T \mathbf{x} \in \mathbb{R}^C$

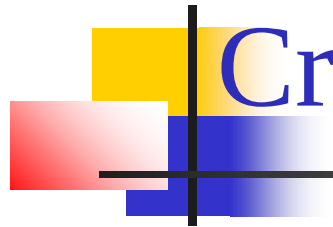
$\mathbf{a} = \text{softmax}(\mathbf{z}) \in \mathbb{R}^C$

$a_i > 0, \sum_{i=1}^C a_i = 1$

short form

$$\mathbf{z} = \text{softmax}(\mathbf{W}^T \mathbf{x})$$

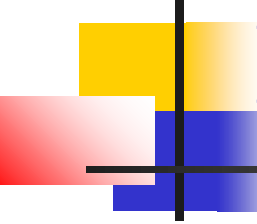




Cross entropy

The cross-entropy between two discrete distribution vectors p and q is defined as

$$H(\mathbf{p}, \mathbf{q}) = - \sum_{i=1}^C p_i \log q_i$$



Loss function

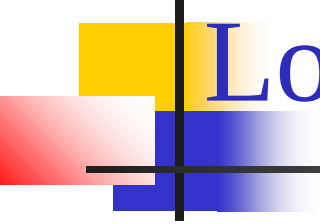
In the case of C classes, the loss between the predicted output and the actual output of a data point $\mathbf{x}_i$ with the **one-hot label** $\mathbf{y}_i$ is given by

$$J_i(\mathbf{W}) \triangleq J(\mathbf{W}; \mathbf{x}_i, \mathbf{y}_i) = - \sum_{j=1}^C y_{ji} \log(a_{ji})$$

$$J_i(\mathbf{W}) = -\log(a_{y_i,i})$$

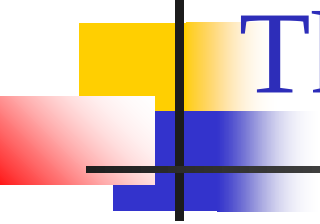
Combining all data pairs $(\mathbf{x}_i, \mathbf{y}_i)$ for $i=1,2,\dots,N$, the loss function for softmax regression is defined as

$$J(\mathbf{W}; \mathbf{X}, \mathbf{Y}) = -\frac{1}{N} \sum_{i=1}^N \log(a_{y_i,i})$$



Loss function for a single data point

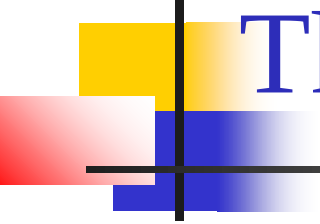
$$\begin{aligned} J_i(\mathbf{W}) &= - \sum_{j=1}^C y_{ji} \log(a_{ji}) = - \sum_{j=1}^C y_{ji} \log \left(\frac{\exp(\mathbf{w}_j^T \mathbf{x}_i)}{\sum_{k=1}^C \exp(\mathbf{w}_k^T \mathbf{x}_i)} \right) \\ &= - \sum_{j=1}^C \left(y_{ji} \mathbf{w}_j^T \mathbf{x}_i - y_{ji} \log \left(\sum_{k=1}^C \exp(\mathbf{w}_k^T \mathbf{x}_i) \right) \right) \\ &= - \sum_{j=1}^C y_{ji} \mathbf{w}_j^T \mathbf{x}_i + \log \left(\sum_{k=1}^C \exp(\mathbf{w}_k^T \mathbf{x}_i) \right) \end{aligned}$$



The derivative of the loss function

- The formula for calculating the derivative

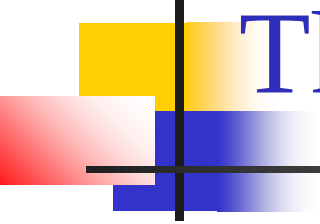
$$\nabla_{\mathbf{W}} J_i(\mathbf{W}) = [\nabla_{\mathbf{w}_1} J_i(\mathbf{W}), \nabla_{\mathbf{w}_2} J_i(\mathbf{W}), \dots, \nabla_{\mathbf{w}_C} J_i(\mathbf{W})]$$



The derivative of the loss function

Gradient with respect to each column of $\mathbf{w}_j$

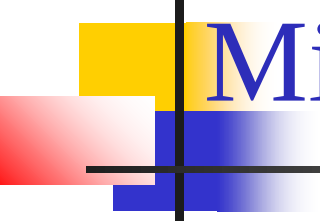
$$\begin{aligned}\nabla_{\mathbf{w}_j} J_i(\mathbf{W}) &= -y_{ji}\mathbf{x}_i + \frac{\exp(\mathbf{w}_j^T \mathbf{x}_i)}{\sum_{k=1}^C \exp(\mathbf{w}_k^T \mathbf{x}_i)} \mathbf{x}_i \\ &= -y_{ji}\mathbf{x}_i + a_{ji}\mathbf{x}_i = \mathbf{x}_i(a_{ji} - y_{ji}) \\ &= e_{ji}\mathbf{x}_i \text{ (với } e_{ji} = a_{ji} - y_{ji}\text{)}\end{aligned}$$



The derivative of the loss function

With $e_i = a_i - y_i$, $E = A - Y$ being the difference between the predicted output and the actual output, we have:

$$\begin{aligned}\nabla_{\mathbf{W}} J_i(\mathbf{W}) &= \mathbf{x}_i [e_{1i}, e_{2i}, \dots, e_{Ci}] = \mathbf{x}_i \mathbf{e}_i^T \\ \Rightarrow \nabla_{\mathbf{W}} J(\mathbf{W}) &= \frac{1}{N} \sum_{i=1}^N \mathbf{x}_i \mathbf{e}_i^T = \frac{1}{N} \mathbf{X} \mathbf{E}^T\end{aligned}$$



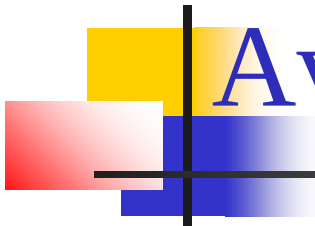
Mini-batch gradient descent

- Denote N_b as the size of each batch

$$X_b \in \mathbb{R}^{d \times N_b}, Y_b \in \{0,1\}^{C \times N_b}, A_b \in \mathbb{R}^{C \times N_b}$$

- Update W

$$\mathbf{W} \leftarrow \mathbf{W} - \frac{\eta}{N_b} \mathbf{X}_b \mathbf{E}_b^T$$



Avoiding overfitting

$$\bar{J}(\mathbf{W}; \mathbf{X}, \mathbf{Y}) = -\frac{1}{N} \left(\sum_{i=1}^N \log(a_{y_i, i}) + \frac{\lambda}{2} \|\mathbf{W}\|_F^2 \right)$$